HEINRICH-HEINE-UNIVERSITÄT DÜSSELDORF

Bachelor Thesis

Implementation and evaluation of the reinforcement learning methods Self-Play and League Training in the MicroRTS environment

Name!!!

Degree:	B.Sc. Computer Science
Supervisor:	Professor Dr. Paul Swoboda
Co-Supervisor:	Dr. Peter Arndt
Faculty:	Faculty of Mathematics and Natural Sciences

XXXXXXXXXXX

Abstract

This thesis improves an existing agent for the real-time strategy (RTS) game MicroRTS. RTS games are considered a challenging domain for Deep Reinforcement Learning (DRL), since agents must deal with large combinatorial action spaces, simultaneous decisions, as well as delayed and often sparse rewards. As a baseline method, we use Proximal Policy Optimization (PPO). In many DRL training setups, such as pure PPO, the agent reaches a point where it cannot be trained further efficiently by using PPO alone, because there are not enough strong opponents to train against or because the available opponents are not diverse enough to develop a robust policy. An obvious and simple countermeasure is Self-Play (SP) however, in practice it can be unstable and lead to overfitting or cyclical strategies, which can reduce performance against external opponents.

League Training (LT) stabilizes SP by building a population of agents across different training stages and roles. The training agent is deliberately matched against diverse opponents. Among others, against specialized agents that were trained to exploit weaknesses of the current policy. We evaluate the resulting agents in matches against various opponent types and compare LT with pure SP as well as with our initial agent. Our results show, that LT increases the robustness and generalization capability of the agent against different opponents.

Contents

1. Introduction

Reinforcement Learning (RL) has established itself in recent years as a powerful approach for developing agents in complex environments. Game environments such as RTS games pose a particular challenge in this regard, with large action spaces, long-term planning and simultaneous decisions. However, they also provide clearly defined state spaces, reward structures and repeatable experiments, which makes them an ideal and challenging test environment for researching RL methods.

MicroRTS. MicroRTS is a simplified version of RTS games that was developed specifically for research [Huang et al., 2021]. It is discrete and reduces the visual and mechanical complexity of large commercial RTS games, such as StarCraft II. At the same time, it retains the core difficulties, for example resource management, unit production, combat mechanics and tactical positioning. MicroRTS is therefore suitable for the systematic investigation of learning methods. MicroRTS also contains numerous training challenges, such as large combinatorial action spaces, delayed rewards, multi-agent interactions, and it also requires robust strategies against different opponents. For instance, the agent must decide early whether it wants to play aggressively or defensively against the current opponent, which affects the overall game strategy and requires long-term planning.

Self-Play-Based Approaches. A central problem in RL is generalization across different opponents and strategies: agents often learn strategies that are strongly tailored to the training opponent or a specific game situation. This leads to overfitting and limited robustness. SP, Fictitious Self-Play (FSP) and LT are approaches that address this problem. In SP, the agent trains by playing against itself, which leads to a continuous learning process that does not require additional diverse or strong opponents. However, training is often unstable because of cyclical learning.

League Training (LT). To reduce the instabilities of pure SP, the agent in FSP is not trained against the current agent but against previous versions of itself, which prevents cyclical learning [Heinrich et al., 2015]. LT extends FSP by training the agent against historical opponents from a league that consists of different agent types, for example policy snapshots from earlier training stages as well as specialized agents [Vinyals et al., 2019]. The goal is to stabilize the learning environment, in which agents compete against a broad spectrum of strategies and thereby become more robust.

Thesis Outline. Despite the theoretical advantages, the practical implementation of LT in complex environments such as MicroRTS is associated with significant challenges, that need to be addressed. These include

- the efficient management and selection of opponents for training the Main Agent and for the specialized agents,
- stabilizing the training process under non-constant opponent distributions,
- dealing with large action spaces and sparse or delayed rewards,
- ensuring opponent diversity through sufficient exploration and preventing overfitting to specific opponents or strategies

In addition, we will compare and evaluate the results emperically. In particular, we will analyze the learning curve, strategic diversity and robustness for the different training approaches.

Training of the Base Agent. The Base Agent used in this project is trained in a three-stage procedure that combines Behavioral Cloning (BC) and BC fine-tuning (BC-FT), followed by fine-tuning with PPO [Huft, 2025]: The two BC phases enable the agent to quickly learn a competent base strategy by imitating expert behavior [Torabi et al., 2018]. Learning from scratch with DRL methods such as PPO can be very time-consuming [Martin and Jhala, 2021]. However this training approach reduces this initial, often inefficient exploration in DRL [Torabi et al., 2018]. PPO is a well-known on-policy policy optimization method that clips the policy update size to stabilize policy updates [Schulman et al., 2017]. PPO further improves the performance of the BC agent through exploration and adaptation to the environment, allowing the agent to adapt and generalize beyond the expert strategies [Martin and Jhala, 2021]. This two-stage approach, in which a policy pre-trained with BC is refined by a DRL algorithm, has proven effective in reducing training time and improving overall performance compared to a pure DRL approach [Martin and Jhala, 2021].

2. Related Work

2.0.1 Game Playing Agents in MicroRTS

The unpublished bachelor thesis by M. Huft, on which this work builds, develops a competitive MicroRTS agent and evaluates the hybrid training pipeline of BC, BC-FT and PPO used to train the Base Agent introduced above [Huft, 2025].

Results and Limitations. The results show a clear performance increase across all three training phases (Table 1): BC mainly learns basic game mechanics, while BC-FT improves generalization against multiple bots. Finally, PPO achieves high win rates against many of the built-in bots [Huft, 2025]. There are still agents in the results against which the Base Agent does not perform well. The thesis also identifies limitations, in particular the focus on a single map and the limited opponent diversity, and proposes extensions using Self-Play or league mechanisms. Exactly this proposal is taken up by the present work by extending the existing Base Agent and the initial code with SP and LT. The opponents against which the evaluation is conducted include the MicroRTS built-in bots (RandomBiasedAI, WorkerRushAI, LightRushAI and CoacAI).

Table 1. Winrates (%) for BC, BC-FT and PPO reported in [Huft, 2025]. The Base Agent in this thesis is trained using the same training pipeline, however these results are of a different Version of the Base Agent.

Bot	BC	BC-FT	PPO
WorkerRushAI	3	50	99
PassiveAI	98	100	100
LightRushAI	22	93	99
CoacAI	0	8	96
Mayari	0	5	95
RandomAI	52	95	99
RandomBiasedAI	18	68	86
rojo	0	34	84
mixedBot	4	44	59
izanagi	0	62	61
tiamat	11	78	84
droplet	0	31	59
guidedRojoA3N	4	42	75
naiveMCTS	0	5	3

2.0.2 MicroRTS-Py (Gym-μRTS)

Huang et al. provide an efficient Gym-compatible interface for our environment MicroRTS [Ontañón, 2013] with Gym-μRTS/MicroRTS-Py [Huang et al., 2021]. MicroRTS-Py is an established and widely used DRL baseline that enables the investigation of DRL approaches in full real-time strategy game scenarios without requiring extensive technical infrastructure such as high-performance computing clusters [Huang et al., 2021].

Proximal Policy Optimization (PPO) Training. The openai/baselines [Dhariwal et al., 2017] implementation of PPO was adapted in order to fit MicroRTS-Py. We use a large part of this implementation in LT [Huang et al., 2021]. It was also previously used in the PPO training phases of the Base Agent [Huft, 2025]. On top of openai/baselines, invalid action masking and suitable action/observation representations (e. g. GridNet [Han et al., 2019]) as well as architectural choices (such as IMPALA-CNN, a deep convolutional neural network structure with residual blocks for processing grid-based state representations in RL) [Espeholt et al., 2018] were added [Huang et al., 2021]. In a modified form, these are still used in the LT code.

GridNet. Huang et al. considered using GridNet or Unit Action Simulation (UAS) instead of GridNet. UAS iterates over all units at each step and returns the possible actions for each unit, which substantially reduces the action space. In contrast, GridNet predicts an action for each cell on the map. This means that GridNet executes its large action space in a single step, but this also simplifies the training [Huang et al., 2021, Han et al., 2019]. Against the built-in bots of MicroRTS, which are based on heuristic rules, the PPO implementation with invalid action masking and IMPALA-CNN achieves an average win rate of 91% with UAS and 84% with GridNet. Ultimately, the decision was made to focus on GridNet, since its lower complexity and its higher compatibility with SP (and LT) [Huang et al., 2021]. SP was also evaluated in the work. However, there was a bug in the code that likely affected the results for SP [Goodfriend, 2024].

2.0.3 RAISocketAI

In 2023 RAISocketAI was the first DRL agent which won the IEEE MicroRTS Competition. RAISocketAI includes a re-implementation of MicroRTS-Py, which is used in our code. Among other things, data generation via replays is an important basis for BC [Huft, 2025] and a bug in the old implementation was fixed that marked non-owned resources as owned resources when the agent is player 2. According to Goodfriend, this likely caused SP in the previous MicroRTS-Py implementation to yield no improvements. The RAISocketAI implementation of FSP achieved an average win rate of 91% against the same MicroRTS built-in bots as in MicroRTS-Py, which is significantly higher than 84% in the previous implementation. However, FSP achieved only a 20% win rate against CoacAI, whereas the best previous implementations were almost always able to defeat CoacAI. With fine-tuning on CoacAI and Mayari, the average win rate against the built-in bots was increased to 98% [Goodfriend, 2024]. LT was not implemented or evaluated in the work.

2.0.4 BeClone

BeClone demonstrated the effectiveness of BC with DRL fine-tuning in MicroRTS-Py using a two-phase training strategy for RTS agents: first, an initial policy is learned from experts via BC, and subsequently an RL fine-tuning is performed using Advantage Actor-Critic (A2C) [Martin and Jhala, 2021].

2.0.5 AlphaStar

AlphaStar is a DRL agent for the complex RTS game StarCraft II, which was developed by DeepMind [Vinyals et al., 2019]. It demonstrates that LT can decisively contribute to robustness in complex RTS domains. The agent in AlphaStar was pre-trained with imitation learning from human replays.

Pre-League Training. In this thesis, we replace imitation learning with the BC phases and the PPO phase. Similar to AlphaStar our agent was further trained in a league of different agent types (Main Agent, Main Exploiters, League Exploiters and historical snapshots (Historical Agents)) to reduce overfitting and forgetting and to promote diversity.

League Composition and Training. As in AlphaStar, in this thesis Main Exploiters are trained by training the Base Agent against all historical versions of the Main Agenten in the league, in order to identify and exploit weaknesses. At the same time, League Exploiters train against all Historical Agents. The Main Agent trains with Prioritized Fictitious Self-Play PFSP against historical versions of itself, Main Exploiters and League Exploiters. The prioritization of the historical opponents is computed based on the Main Agent's win rate against the respective Historical Agents. In this thesis, the prioritization formula was adjusted to better fit the MicroRTS environment. An exploiter is added to the league as a Historical Agent only when it shows a high win rate against all opponents or when a predefined step limit is reached [Vinyals et al., 2019].

Differences from AlphaStar. Although MicroRTS and StarCraft II substantially differ in complexity and representation, AlphaStar is an established reference design for stabilizing and generalizing SP through league-based matchmaking mechanisms and the inclusion of specialized exploiters [Vinyals et al., 2019]. AlphaStar performs policy updates using an actor-critic method with an off-policy correction mechanism called V-trace [Espeholt et al., 2018, Vinyals et al., 2019]. This approach is better suited for LT than PPO due to its robustness to off-policy data and its typically better scalability in large distributed setups. However, the implementation of PPO in MicroRTS-Py is significantly simpler and faster to realize, which is why PPO is a practical choice for implementing LT [Vinyals et al., 2019]. In addition, AlphaStar uses TD(λ) [Sutton and Barto, 1998] for value-function estimation.

3. Methods

3.1. MicroRTS-environment

MicroRTS is a two-player zero-sum game that is played on a two-dimensional grid (the map). Different units can be located on each cell of this map and players can execute actions to move/train/attack units and to collect resources. The different units types are:

- **Base:** 

The base can store resources and train workers. It is very important, since it is the only way to store resources. (cost (in resources): 10, hit points (hp): 10)
- **Resources:** 

There is only one type of resource. It can be stored by workers in the base (Number of resources in a cell: depends on the map in our case 25).
- **Worker:**  workers can collect resources and build barracks (cost: 1, hp: 1, attack strength (at): 1).
- **Barracks:**  barracks can train light, heavy and ranged units (cost: 5, hp: 4).
- **Light:**  light units can move quickly, are trained quickly and cost little (cost: 2, hp: 4, at: 2).
- **Heavy:**  heavy units can move slowly are trained slowly and cost a lot, but have high at and hp (cost: 3, hp: 8, at: 4).
- **Ranged:**  ranged units have an attack range of 3 cells (cost: 2, hp: 1, at: 1).

Units of type worker, light, heavy and ranged can move one cell in one of four directions. Units of player 1 are displayed with a blue outline, whereas units of player 2 are outlined in red. Movement is visualized with a gray line, attacking with a red line and gathering with a green line.

Game Setup. Most games are decided within 2000 steps. Therefore, the maximum number of steps is set to 2000. The experiments in this thesis were conducted using the RAISocketAI re-implementation of MicroRTS and the map `basesWorkers16x16A`, which is shown in figure 1. This symmetric map starts each player with one base, five

resources, two workers and 50 resources located near the base [Goodfriend, 2024].

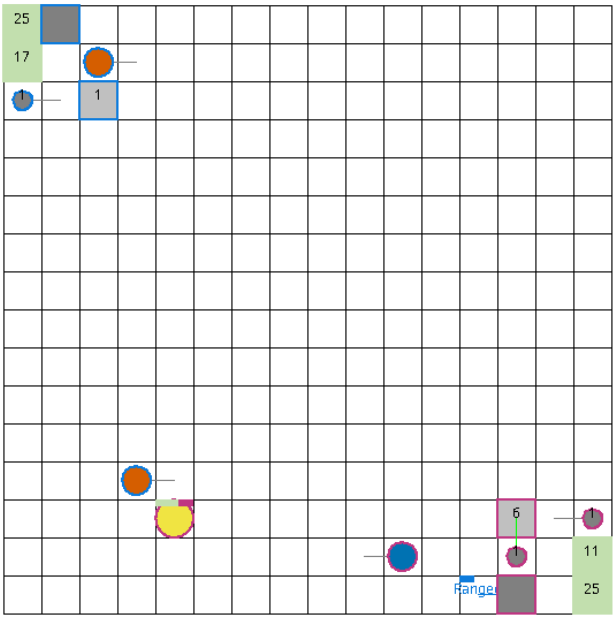

Figure 1. Screenshot `basesWorkers16x16A` map [Huft, 2025].

Action Space. The action space used in this thesis is implemented using the combination of GridNet and invalid action masking [Huang et al., 2021]. GridNet produces a unit action (an action for one specific cell) for each cell of the map, regardless of whether a unit is located on the cell or whether the action is valid [Han et al., 2019]. Building on this, invalid action masking is used, thereby also masking actions of empty cells [Huang et al., 2021]. The unit action space is defined as an 8-dimensional vector per cell:

$$(\text{pos}, \text{type}, d_{\text{move}}, d_{\text{harvest}}, d_{\text{return}}, d_{\text{produce}}, \text{pType}, \text{relPos})$$

The components of the unit-action vector are defined as follows:

- **pos:** linearized index of the cell in the grid (i.e., the grid represented in a one-dimensional space (e. g. for a 16×16 grid: range: [0, 255])),
- **type:** action type (range: [0, 5], 0: NOOP, 1: Move, 2: Harvest, 3: Return, 4: Produce, 5: Attack),
- $d_{\textbf{move}}, d_{\textbf{harvest}}, d_{\textbf{return}}, d_{\textbf{produce}}$**:** directions (range: [0, 3], 0: North, 1: East, 2: South, 3: West) for the respective actions,
- **pType:** type of the unit to be produced (range: [0, 6], 0: resource, 1: base, 2: barracks, 3: worker, 4: light, 5: heavy, 6: ranged),
- **relPos:** relative attack position (range: [0, 48]).

For our agent, the unit action space (without the position dimension) is linearized and represented using one-hot encoding. The position is implicitly encoded by the linear grid indexing of the GridNet output [Huang et al., 2021]. Given a map of size $h \times w$, the action space has shape $h \times w \times 78$. For our 16×16 map, the action space per environment in the GridNet representation would therefore contain $16 \times 16 \times 78 = 19968$ actions. However, many of them can be invalid and are masked by invalid action masking [Huang et al., 2021]. Before applying the invalid action mask, the position is added to each action as a fixed position index. Otherwise, masking would remove the grid structure of the action space, resulting in the removal of the implicit encoding for the position. Depending on the action type, only parts of the action vector are relevant (e. g. if the action type is `Move`, only the components `pos`, `type` and $d_{\texttt{move}}$ are relevant). The other components are ignored.

Observation Space. Observations are represented as a grid of shape $h \times w \times c$ per environment, where h and w are the height and width of the map and c is the number of encoded features of each cell in the game state (e. g. unit types, hit points, move direction, etc.) [Huang et al., 2021]. The screenshot of the map `basesWorkers16x16A` 1 visualizes the observation space without any additional information.

3.2. Training Pipeline

The training pipeline extended in this thesis consists of four phases: BC, BC-FT, PPO and LT (or SP/FSP). The first three phases were already implemented in the previous work [Huft, 2025] and are only briefly described here. In BC/BC-FT, demonstrations from experts (here, the built-in bots CoacAI and Mayari) are used to train an initial policy to learn basic game mechanics/strategies [Huft, 2025].

- The BC phases reduces the initial, often inefficient exploration in DRL methods such as PPO, which can be very time-consuming when trained from scratch [Torabi et al., 2018, Martin and Jhala, 2021].
- The third phase improves the generalization and performance of the agent through additional exploration beyond expert demonstrations [Martin and Jhala, 2021]. PPO constrains the size of policy updates in order to achieve stable training [Schulman et al., 2017].
- Using LT, we aim to further generalize the Base Agent by training PPO not only against the built-in bots of MicroRTS (against which the Base Agent already achieves a high win rate), but also against opponents of

equal strength or stronger. However, many opponents at this level of performance are not suitable for training, since they are very slow and thus slow down the collection of rollouts for PPO. In addition, the agent's generalization capability improves only to a limited extent when training against only a few new opponents, if the training environment does not provide sufficient opponent diversity. LT address both problems by providing numerous opponents that continue to evolve over the course of training and achieve a high win rate against historical versions of the current agent.

3.3. Self-Play (SP)

In SP, the agent trains against its own current version, which leads to a continuous learning process with a co-evolving opponent [Silver et al., 2018].

Implementation. To implement SP in the MicroRTS-Py reimplementation Huang et al. implemented the environment to output the observations (obs) and rewards for both players, which allows the agent to train against itself by simply using the same policy for both players [Huang et al., 2021]. In RAISocketAI, that approach was improved and fixed [Goodfriend, 2024]. In this thesis, the same approach with some adustments is used to implement SP, which is also used as a basis for FSP and LT.

Problems.

- The training is often unstable because of cyclical learning. For example, if policy B performs well against A, C performs well against B and A performs well against C, the agent may be trapped in a cycle in which it repeatedly relearns only the policy that performs well against itself, but not against other opponents [Vinyals et al., 2019].
- A big problem in the SP training setup is that the agent has to learn to play as player 2 as well as player 1. This complicates the training process and can lead to a slow learning process [Huang et al., 2021]. The biggest difference between player 1 and player 2 is that player 1 starts at the top left of the map, whereas player 2 starts at the bottom right. This means that the agent has to learn to start play in two different areas of the map, which can be difficult and time-consuming. The problem is further exacerbated by the fact that the training of this thesis does not start from scratch. Rather it starts from a Base Agent that was only trained as player 1. Therefore, the agent would need to unlearn its bias towards starting at the top left of the map and learn to start at the bottom right as well.

Solutions. FSP/PFSP (3.5) are used to stabilize training and reduce cyclical learning. To address the second problem, the observations for player 2 are transformed by rotating them along by 180 degrees. From the agent's perspective, player 2 also starts at the top-left of the map. This makes the observations for player 1 and player 2 more similar and therefore makes SP with a pre-trained Base Agent possible, even though the Base Agent was exclusively trained as player 1 (Figure 8). Actions are transformed accordingly, as a result the environment still executes the intended actions for player 2. Masks must be transformed as well, since they are based on the environment's observations. The same rotation used for the observations is also applied to the invalid action masks. After masking, the selected actions are transformed back to fit the MicroRTS-Py environment. This includes a position component that is added before masking, since masking removes the implicit positional structure of the GridNet action space.

3.4. Fictitious Self-Play (FSP)

To address the instabilities of pure SP, in FSP the agent is trained not exclusively against the current policy, but against a uniformly sampled pool of the agent's historical policies. This results in the learning-agent learns an effective response to the averaged opponent strategy, reduciing cyclical learning. Under standard assumptions, FSP can converge towards an approximate Nash equilibrium in two-player zero-sum games (with respect to the average strategies). Heinrich et al. also demonstrate this empirically in poker experiments [Heinrich et al., 2015].

Implementation. The implementation is similar to SP, but instead of player 2 always using the current policy, player 2 is sampled uniformly from a pool of Historical Agents (i. e. snapshots of the Main Agent at different training stages).

Problem with the Implementation. If we implement in the described way, a problem arises: Player 1 has priority in conflict resolution (e. g. when both players want to move to the same cell). This gives player 2 a disadvantage, although it has only a minor effect on the results. The actual problem is that the agent can exploit this prioritization in FSP if it learns that optimizing the training objective can be achieved primarily by improving player 1, while deliberately performing poorly as player 2. After sufficient training, the agent will learn a strategy that is based on player 1 being prioritized. Training then stagnates, since the agent already has a high win rate against itself without developing a robust strategy against other opponents.

Example. The agent could learn to block an opponent in front of the base by issuing a simultaneous move to the same target cell. In FSP, player 2 would then be blocked, while player 1 would not, allowing the agent to achieve a win rate close to 100% against itself.

Relevance for LT. In LT, the result of the player 1 priority is not quite as problematic, because there are exploiters that can be trained against, which remain adversarial to the Main Agent. However, a large portion of the training becomes ineffective and provides poor learning signals if the agent exploits the player 1 priority. On top of that, this problem is particularly relevant for LT, which combines SP and FSP, because only Historical Agents are available for training in FSP, making the feedback for player 2 very sparse. In LT the Main Agent can directly train against the current Main Agent. This yields more immediate feedback on the Main Agents performance as an opponent (player 2). As a result, the agent no longer has to plan as far ahead in order to exploit this priority and minimize its own win rate as player 2.

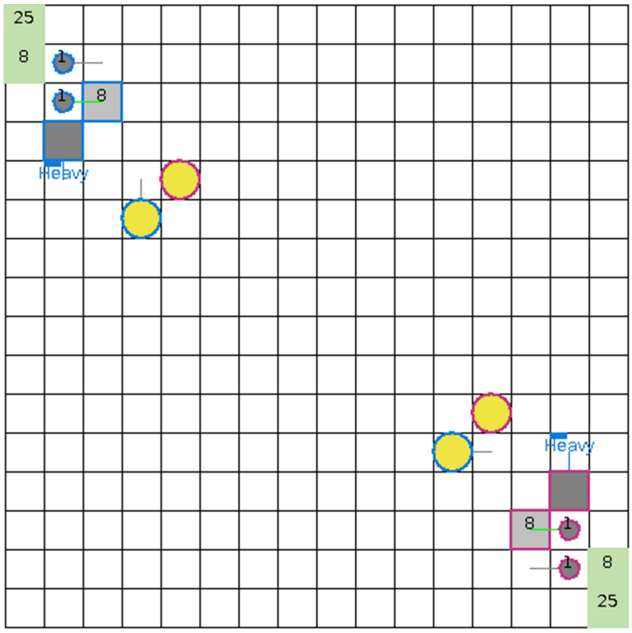

Figure 2. Screenshot showing an action conflict in the game. Both heavy units of both players wanted to move to the same cell, as the opponent's heavy units.

Solutions. In the re-implementation of MicroRTS-Py, there is a mode that resolves conflicts in a randomized manner. However, this does not help in this case, because even before the conflict arises, invalid action masking masks out the action of player 2 since it is invalid. To solve this problem, the learning agents in the parallel games are alternated between playing as player 1 and player 2. This does not prevent the prioritization of player 1. However, it can be shown empirically that the prioritization of player 1 has no major impact on the results by evaluating an agent against itself. Many evaluations were conducted in which the agent played against itself, for example one of the evaluations resulted in a win rate of 38% for player 1, 28% draws and 34% wins for player 2 over 200 evaluation games, which shows that player 1 did not have a substantial advantage. This prevents the exploitation of the prioritization by the learning agent, since it can now also be player 2 and therefore a strategy that is only effective for player 1 is no longer beneficial.

3.5. Prioritized Fictitious Self-Play (PFSP)

LT uses an extended variant of FSP called PFSP [Vinyals et al., 2019]. PFSP extends FSP by selecting opponents against which the agent can train efficiently, rather than selecting opponents uniformly. The selection probability is based on the win rate of the learning agent against the potential opponent [Vinyals et al., 2019]. As in AlphaStar, the PFSP win rate in this thesis is calculated by considering a draw to be 50% of a win for the learning agent. Win rates outside of PFSP are calculated without considering draws.

Prioritization Function. The exact prioritization function can differ for different agent types. In AlphaStar, for the Main Agent opponents of medium difficulty (PFSP win rate close to 50%) are prioritized. While exploiters prioritize the strongest opponent (against which the PFSP win rate is closest to 0%). Adapted prioritization functions tailored to our training setup are used in this thesis, which deviate from those used in AlphaStar. An important difference is that training with an opponent against which the Main Agent has a win rate (without draws) close to 0% yields little informative gradient or learning signal.

Implemented Prioritization Function. In the final LT setup, the Main Agent uses `focused_strong_boosted_with_draws`, while Main Exploiters and League Exploiters use `focused_strong_with_draws`. To avoid prioritizing opponents that provide little learning signal, the following prioritization functions was used for the exploiters:

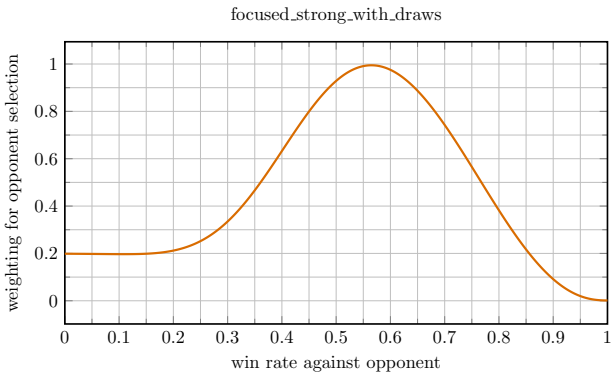

Figure 3. Prioritization function for the exploiters in PFSP.

As can be seen in the figure 3, we prioritize opponents with a PFSP win rate around 50%. That is because most of our matchups are very draw-heavy. Therefore, it is likely that the win rate (without draws) is close to 0%, when the PFSP win rate is below 50%. As a result, such opponents would otherwise be selected more frequently despite providing only a very weak learning signal. Furthermore, opponents with a PFSP win rate of 0% are prioritized over those with a PFSP win rate of 100%, since there is substantial learning potential for an opponent with a PFSP win rate of 0%, whereas a PFSP win rate of 100% usually means that the agent already defeats the opponent reliably. In this thesis PFSP was modified in a way that all agents have a non-zero probability of being selected.This ensures that no opponent is completely ignored because it lost all initial games (yielding a 100% PFSP win rate for the learning agent), since these initial results may have been outliers.

3.6. League Training (LT)

LT combines and extends SP and PFSP to further stabilize training. It also promotes robustness by training against opponents sampled from a league, including Main Exploiters and League Exploiters and Historical Agents [Vinyals et al., 2019].

League Composition and Roles. Learning agents of the league were trained in parallel. Similar to AlphaStar [Vinyals et al., 2019], the league in this thesis consists of the following agent types:

- **Main Agent:** The primary policy that is optimized and evaluated.
- **Historical Agents:** Frozen snapshots of earlier Main Agents or exploiter agent policies. They stabilize training by providing a distribution of past strategies and mitigate catastrophic forgetting.
- **Main Exploiters:** Agents trained only against historical Main Agents, which encourages them to identify and exploit weaknesses in the Main Agent.
- **League Exploiters:** Agents trained against all Historical Agents, which further promotes diversity by identifying systemic weaknesses of the league.

Historical Agents are the only agent type that is not a learning agent. The main differences between the roles are the opponent-sampling distribution and the hyperparameters. All agents share the same observation/action representation (GridNet with invalid action masking) and use the same underlying PPO algorithm, but they are trained with role-specific hyperparameters (e. g. learning rate and PFSP prioritization function).

Training Loop and Snapshot Management. LT iterates through cycles consisting of the following phases:

- **Select opponent:** sample an opponent from the league,
- **Collect rollouts:** run matches between the learning agent and the sampled opponent for a fixed number of steps, no agent is updated during this phase,
- **Update policy:** perform PPO updates using the collected trajectories against the sampled opponent,
- **Update statistics:** periodically evaluate against a subset of league members to update win rate estimates used by PFSP,
- **Add snapshots:** at predefined intervals (or when performance criteria are met), store a frozen copy of the current agent as a new Historical Agent.

A phase of the training loop is considered completed only after every agent type (Main Agent, Main Exploiters, League Exploiters) has completed the phase. For example, the Main Agent only proceeds to the next phase after all Main Exploiters and League Exploiters have completed the current phase.

Bot Environments. Bot environments are added to the league to stabilize training by continuing the PPO training against the built-in MicroRTS bots CoacAI and Mayari, which the Base Agent was trained against. Only the Main Agent trains against the built-in bots, since learning signals from built-in bots could interfere with identifying and exploiting weaknesses in the Main Agent or the league, which is the main purpose of the exploiters (e. g. if the built-in bots are too strong against an exploit that the Main Agent has not yet trained against). In MicroRTS-Py the environments for built-in bots and for SP are not interchangeable. This is because the built-in bots are not implemented as neural policies and therefore cannot be used

in SP/FSP/LT directly. The number of parallel environment instances and their built-in bot opponent (if any) have to be defined at initialization of the MicroRTS-Py environments, which means the number of environments for each built-in bot remains fixed. One of the main advantages of LT is that training against opponents with bad learning signals is reduced. To retain that improvement we want the number of bot environments to be dynamic. Therefore, the number of bot environments is adjusted dynamically by reinitializing a MicroRTS instance, which is only used for training against the built-in bots and not for SP/FSP/LT (because reinitialization also means resetting the environments in the MicroRTS instance). This MicroRTS instance for built-in bots executes its step function at the same time as the MicroRTS instance for SP/FSP/LT.

Number of Parallel Bot Environment Instances. Two hyperparameters define the maximum number of parallel bot environment instances and the minimum number of parallel bot environment instances. Among other things, references to the observation, action and invalid action mask buffers must be adjusted to the correct size and partially reset whenever the number of parallel bot environment instances is changed.

Summary: Opponent Selection. PFSP samples the next opponent from a given set of possible opponents, with probabilities based on the current learning agent's win rates against the potential opponents, whereas SP always trains against the current learning agent. The opponent selection for LT is illustrated in Figure 7. Depending on the agent role, opponents are either the current Main Agent or sampled from specific subsets of Historical Agents using PFSP. The Main Agent also trains against additional bot environments, while Main Exploiters and League Exploiters are matched only against agents from the league.

Resetting Exploiters. The AlphaStar paper [Vinyals et al., 2019] describes that exploiters reset their weights to the initial values (here: the Base Agent). In our approach exploiter resets also reset the optimizers, game statistics (e. g. win rates so that PFSP does not use outdated information) and the unit bonus distribution 3.7 (if applicable) that is later explained in this thesis. Additionally, exploiters only reset after a game has ended. As a result, an exploiter can reset during a rollout. If that happens, the rollout contains trajectories from the old policy and therefore cannot be used for the on-policy training in our PPO implementation. To keep the training as simple as possible, the entire PPO update for the rollout after an exploiter reset is skipped.

AlphaStar LT Pseudocode. In addition to the modifications made to adapt the LT pseudocode to the MicroRTS environment the following corrections to the AlphaStar pseudocode [Vinyals et al., 2019] were made:

- When resetting exploiters, the pseudocode first resets the weights and only then saved the snapshot for the Historical Agent. In our implementation, this order is reversed so that the snapshot is not taken from the reset exploiter.

- Main Exploiters were only trained against the current Main Agent, if they had a high win rate against the Main Agent from the very beginning. As soon as this win rate dropped, the Main Exploiter stopped training against the Main Agent for the remainder of this reset cycle. However, in the pseudocode Main Exploiters were only reset/checkpointed (creating a historical Main Exploiter) if they had a sufficiently high win rate against the Main Agent. This means that a Main Exploiter would never reset if it initially had a low win rate against the Main Agent. Since the current Main Exploiter has already trained against the Base Agent, it is very likely that it will never checkpoint before the maximum number of steps is reached, even if it has a high win rate against all historical Main Agents. To solve this problem, the win rate used for resetting or checkpointing a Main Exploiter is defined as

$$w_{\text{reset}} = \max\left(\min_i w_{\text{hist},i},\ w_{\text{main}}\right),$$

  where $w_{\text{hist},i}$ denotes the win rate against historical Main Agent i and w_{main} the win rate against the current Main Agent. This ensures that the Main Exploiter resets and checkpoints as soon as it achieves sufficiently high win rates against the available historical Main Agents (making high win rate against the Main Agent likely) and no longer has much to learn from the available opponents.

- Another change to the pseudocode is that win rates are set to 0.5 for the initial games. This makes it less likely that the agent will ignore opponents it has not yet played against because of outlier results with very high or very low win rates, including the Main Exploiter versus Main Agent matchup.

3.7. Additional Modifications for Diversity

The biggest problem with the training setup is the lack of opponent diversity. To make exploiters more diverse, exploration can be increased with hyperparameters such as the learning rate, the entropy coefficient and the Kullback-Leibler (KL) regularization coefficient.

Problems with Overall Increased Exploration. Excessively increasing exploration of exploiters can lead to worse performance and longer training times. We hypothesize that is because the starting point (the Base Agent) is already very strong and effective strategies differ substantially between unit types, so that the agent must unlearn a very large portion of the current strategy before it can learn new strategies for other unit types. In between the unlearning and learning phases, the agent performs very poorly, which leads us to believe that there are very distant clusters (corresponding to different unit types) of local maxima in the reward landscape. This could explain why finding different maxima with the same unit type is possible. But changing to a strategy for different unit types is very difficult and time-consuming. Furthermore, the same problem occurs for every exploiter, which compounds the problem. What makes diversity even more difficult is that, in a complex environment such as MicroRTS, league training will most likely not cover all possible Main Agent weaknesses that can be exploited by an exploiter using heavy units (the unit type strongly favored by the Base Agent). This suggests that, even after training, there will remain weaknesses that are easier to exploit than those that require shifting the focus to a different unit type. For example, if the Base Agent, which favors heavy units, is to exploit a weakness that occurs when the Main Agent is attacked by many worker units, then throughout the episode, every worker on the field must refrain from building barracks, since this would waste too many resources for the strategy to work. Instead of using only two workers, the agent has to produce as many as possible, while every unit except two rushes the opponent. The agent only receives a positive reward if all of this happens in the same game, making this behavior unlikely to occur. The second strategy in the example is simmilar to a built-in bot strategy of MicroRTS called WorkerRushAI.

Production Exploration Bonus. To further increase exploration without increasing training time too much, an exploration bonus targeting the production of workers, ranged, heavy and light units can be added to the reward. By targeting the production of different unit types, the agent is encouraged to explore different strategies for different unit types, which can lead to more diverse exploiters. This was achieved by using a mask that gives barracks and bases (units whose only function is to produce other units) an entropy bonus, which encourages more diverse production decisions and thereby increases exploration across different unit types, by making the agent interact with them. While this helped to increase exploration of a broader range of unit types, as reflected by the fact that the agent uses a wider variety of units, it was not enough to make the agent explore strategies for different unit types.

Unit Bonus. To address the problem of insufficient diversity, a clipped unit bonus can be added to the reward, encouraging the agent to use different unit types (e. g. by giving the Base Agent a bonus every time it produces a ranged unit). The number of other units types increased when using the unit bonus. However, even with weaker agents such as the agent after the BC phase and over 20 million steps, the agent did not develop strategies centered on different unit types and instead produced the unit type with a bonus at the end of a round, when the result was already decided.

Unit Bonus Distribution. Unit bonus distribution is an improved method for increasing the diversity of exploiters based on the unit bonus. The idea is to continue training the agent with a new feature that receives the unit bonus sampled from a distribution that is updated every game. The trained agent can be used as an exploiter with a static bonus or a distribution-based bonus, resulting in an exploiter with definable tendencies toward different unit types. This also allows the retraining of the BC and PPO phases with the unit bonus, because only one agent needs to be trained instead of multiple agents with different fixed unit bonuses, which would be very time-consuming. Retraining the agent from scratch with the unit bonus distribution showed promising results. However, the training was not continued long enough to evaluate the effect of the unit bonus distribution on the diversity of the exploiters, since training with the unit bonus distribution is very time-consuming because the BC phase did not work with the unit bonus distribution method and therefore had to be skipped. Continuing the training with the Base Agent showed the same problem as training with the unit bonus.

Annealed Entropy Coefficient. The annealed entropy coefficient decreases the weight of the entropy bonus in the loss function and thereby decreases the exploration of the agent over the course of training. This was implemented for the exploiters to further increase exploration during the early training steps, while retaining a more deterministic policy in the later stages of training. This coefficient resets when the corresponding exploiter resets. In the case of a League Exploiter, it is possible that the exploiter checkpoints but does not reset. If that happens, the annealed entropy coefficient is reinitialized to half of its previous initial value.

New Initial Agents. To mitigate the insufficient opponent diversity, we created new initial agents that focused on different unit types and can be used as alternative initial agents for the Main Exploiters in LT. This enables the exploiters to take advantage of weaknesses related to different unit types without first having to unlearn the

strategy for the unit type favored by the Base Agent. This can lead to more diverse exploiters in fewer training steps. We did not created new initial agents for the Main Agent because we prioritize further training the already strong Base Agent. The new initial agents were not trained long enough to be as strong as the Base Agent, but they are strong enough to identify weaknesses in the Main Agent from a previous run that did not use the new initial agents and thereby contribute to improved generalization.

3.8. Delta Score.

To better guide the agent in the sparse reward landscape of MicroRTS, we added the change in the MicroRTS score between two consecutive time steps to the reward [Huang et al., 2021, Ontañón, 2013]. The score for each environment instance is calculated based on the current game state, which includes the number and type of units, their hit points and the resources of both players. The delta score is the difference between the score at the current time step and the score at the previous time step, which makes it a usefull immediate learning signal for the agent. The score does not always accurately predict how well an agent is playing, which is why a coefficient is used to scale the delta score, so that the accumulated weighted delta score over a game is much smaller than the win/loss reward. In the prior work by M. Huft [Huft, 2025], the agent we now use as the Base Agent was trained with an attack bonus, which is a reward for attacking the opponent's units. The attack bonus was removed for our training and we adjusted the per-step delta score in the early stages of training to be approximately as large as the per-step attack bonus used in earlier training runs.

4. Experiments

We use the Base Agent from the prior work of M. Huft [Huft, 2025], which was trained by using a combination of BC and PPO, as the initial agent in this thesis unless otherwise specified. The map used is `basesWorkers16x16A`, a symmetrical map also used in related work such as [Huang et al., 2021, Huft, 2025, Goodfriend, 2024]. The Base Agent is trained in LT with (???)4 Main Exploiters, (???)1 League Exploiter, with (???)25 environments each, for approximately (???)130000000 steps. The Main Agent played around (???)4500 games against the built-in bots CoacAI and Mayari. The win rates against both bots over the global training steps were monitored during training and shown in Figure 4. The bot environments were evenly split between the two bots, with a maximum of 10 and a minimum of 5 environments for both bots combined. The number of bot environments throughout training is shown in Figure 5. This results a total of (???)160 parallel environments for LT when all bot environments are active. It requires a lot of computational resources, especially since the observations are not purely boolean and therefore required more memory. The peak memory usage increases by approximately 2500 MiB for an agent with 25 parallel environments. However, it is important to keep the number of environments per agent above a certain threshold, because the number of environments affects the rollout size and a too small rollout can lead to unstable training.

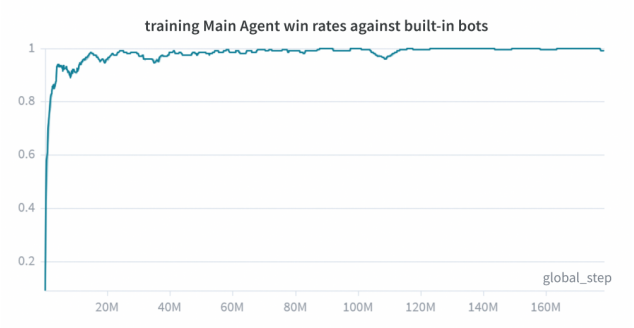

Figure 4. Win rates of CoacAI and Mayari against the Main Agent over the course of training.

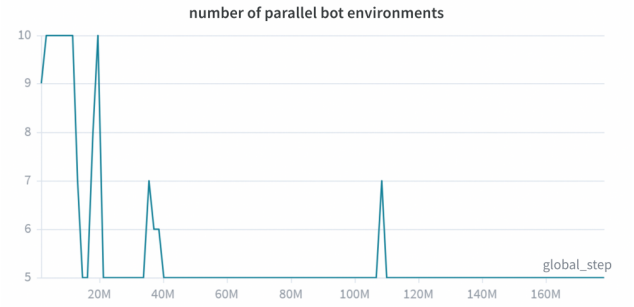

Figure 5. Number of parallel bot environments over the course of training.

4.1. Hyperparameters

Notable hyperparameters for the Main Agent include a higher learning rate of $5e$-5 than the Base Agent, namely $2.5e$-5, as well as higher value function coefficients of 0.15 for the Main Agent and 0.1 for the exploiters. An annealed entropy coefficient is used, starting at 0.025 and ending at 0.0075 for the Main Agent and 0.02 for exploiters. The annealed entropy coefficient for exploiters is often reset in the exploiter resets, as explained in 3.7. After sufficient steps, the entropy coefficient for exploiters remaies high because of repeated resets, which helps them find exploits faster, while the annealing helps them retain and further improve these exploits. The entropy coefficient is very

high in comparison to that of the Base Agent, which uses a constant coefficient of 0.005. This was mostly done to overcome the strong preferences of the Base Agent, which can make it more difficult for the agent to change its strategy and explore. All hyperparameters used are listed in Table 8.

4.2. Evaluation

We evaluated the Main Agent against other agents, including the built-in bots and the Base Agent, across at least 100 games per opponent. Table 3 lists the evaluation results of the Base Agent for comparison. The results vary from those reported in the thesis of M. Huft [Huft, 2025] because the original Base Agent from the prior work could not be found. Therefore a different version of the Base Agent provided by M. Huft, but trained using the same procedure, was used. Table 4 presents the evaluation results of the Main Agent.

Checkpoint and Reset Statistics. There were (???)7 checkpoints for the Main Agent, (???)9 checkpoints for the Main Exploiters and (???)2 checkpoints for the League Exploiter. Table 2 presents the number of checkpoints for each agent type and the approximate total games that exploiters trained with them.

4.3. Results

The agent learned to keep the base defended, even while attacking with other units, which is a clear improvement over the Base Agent, which often leaves the base undefended while attacking. As shown in Tables 3 and 4, LT improves the agent's performance against almost all evaluated opponents. The differences between the win rates are listed in Table 5 for better comparison. The win rates against the bots used in training reach 100%, which is a major improvement over the Base Agent, which achieves a 96% win rate against CoacAI, but only a 67% win rate against Mayari. The results of our Base Agent differ substantially from the results reported in the thesis of M. Huft [Huft, 2025]. However, the Main Agent still shows improvement over the results reported in that thesis, which report a 96% win rate against CoacAI and a 95% win rate against Mayari. The trained Main Agent also performs well against the Base Agent against which it was trained, achieving a win rate of 89%. The win rate against RandomBiasedAI is another major improvement, increasing from 79% to 97%. This indicates that, in comparison to the Base Agent, the trained Agent performs substantially better against bots that try to draw the game instead of win the game. The agents in the section "Other Bots" are considerably stronger against the Base Agent than the built-in bots, which is reflected in the lower win rates against them. However, the Main Agent still shows improvements over the Base Agent against most of these opponents. Highlights include the win rate against Rojo, which improves from 81% to 100%, while the win rate against MixedBot also increases from 43% to 57%. The win rate against Izanagi falls from 72% to 70%, however the loss rate improves from 16% to 7%, suggesting that the Main Agent defends the base more effectively while attacking. WorkerRushAI is the only opponent against which the Main Agent performs notably worse than the Base Agent. This shows that the Main Agent still has weaknesses that were not found.

Main Agent without Unit Focus Evaluation. Table 6 presents an example evaluation for a Main Agent trained without unit focus. As can be seen in the table, the Main Agent with unit focus performs better in all matchups. The biggest differences are the win rates against Droplet, which improve with the unit focus from 45% to 62% and GuidedRojoA3N, which improves from 64% to 87%.

PPO Finetuning. After the LT training, the Main Agent was further trained with PPO for approximately (???)15 million steps. The 25 environments were evenly split between CoacAI, Mayari and WorkerRushAI. The learning rate was set to 1.5e-6, the entropy coefficient was set to 0.005 and no exploiters were used during this finetuning phase. The results are presented in Table 7. After finetuning, the win rate against workerRushAI is now 100% and most other win rates also improved. The averaged win rate against all evaluation opponents excluding the Base Agent is 77% for the Base Agent. After LT and PPO finetuning, this averaged win rate increases to 84%, which is a substantial improvement of 7%. If we also exclude the built-in bots, which already have a win rate of approximately 100% for the Base Agent, the averaged win rate against the remaining opponents increases from 72.6% to 82.5%, which is an even bigger improvement of almost 10%.

4.4. Discussion

LT improved the Main Agent's win, draw and loss rates against a wide range of opponents, including the built-in bots and the Base Agent used during training and generalized to other bots that were not used in training. This can improve the Main Agent's performance further than PPO could by itself, without the agent overfitting to the opponents used in training even after many different training sessions and further PPO training [Huft, 2025]. The possible interpretation is that pure PPO does not search for well-generalizing policies. Instead, the training optimizes performance against the limited set of training opponents. It therefore may pass only briefly through policies that generalize well, before continuing towards more specialized solutions. As a result, within a single run, the PPO may

only traverse a limited subset of the policies with good generalization that are in principle reachable. In contrast, LT changes the training opponent distribution such that robust performance against diverse opponents becomes part of the optimization target itself. This makes it more likely that training explores and retains a broader range of policies with good generalization. The results show that LT was able to discover policies that generalize well while also outperforming the Base Agent even against its original training opponents despite being trained against a larger set of opponents.

Exploration: Effects of New Initial Agents. It was more difficult than expected to generate diverse exploiters because of the long transitions without reward between strategies for different unit types, which is why the introduction of new initial agents was particularly important. The new initial agents contributed to generalization across diverse opponents. Table 6 shows that performance improved against a wider range of opponents compared to the previous evaluation that did not use the new initial agents. The most notable difference is that, during the training without the new initial agents, no opponents similar to WorkerRushAI developed, resulting in a very low win rate against WorkerRushAI, while the new Main Agent as well as the Base Agent have no difficulty defeating WorkerRushAI. This lack of opponent diversity is also seen in the win rates against Droplet, which also has a different strategy than the Base Agent.

Despite the improved diversity and general performance of the Main Agent, there are still some limitations of the training approach:

- **Limitations: Exploiter Diversity.** The results show that the exploiters are able to exploit the Main Agent while staying within their respective unit type focus. This suggests that exploiter diversity is sufficient to identify and exploit weaknesses within the set of strategies for the respective unit type favored by the Base Agent. A weakness of the training approach is that exploiters would need a very long time to discover strategies centered on different unit types, because transitions between such strategies often involve long reward-free phases. This is why the new initial agents, with a focus on different unit types, were implemented, allowing the exploiters to find exploits related to different unit types without first having to unlearn the strategy for the unit type favored by the Base Agent. However, it is not proven that the weaknesses for other unit types are the only weaknesses of the Main Agent that require hard-to-reach exploiter states, which are unlikely to be represented by a historical exploiter after training. At the same time, the final unit composition in Figure 6 for the worker focused initial agent shows that at least some mixed strategies can be achieved with this training setup. For example, the agent focused on worker units learned to also frequently use ranged units, even though it was not initially trained with them. With this, we have initial agents for each unit type for the exploiters, as well as examples of effective exploits within the set of strategies for the focused unit types. On top of that, we also have multiple other examples in different runs, that show the exploiters' capability to explore strategies with mixed unit types, which seems like the strategy type most likely to be similarly hard-to-reach. This suggests that the exploiter set is diverse enough to cover many relevant strategy variations and improve the performance of the Main Agent, even against many other agents that were not included in the training or evaluation.
- **Limitations: Computational Resources.** LT also requires a lot of computational resources. Our setup has a maximum of 160 parallel environments, while pure PPO only uses 24 parallel environments [Huft, 2025]. This causes a much higher memory usage and long training times.
- **Limitations: Map Constraints.** This thesis, just like the thesis for the Base Agent [Huft, 2025], only uses one map (`basesWorkers16x16A`), with full observability of all units and resources. This is a very specific setting.

Exploiter Diversity in AlphaStar. [Vinyals et al., 2019] LT mitigated the SP problem of cyclic training. However, a related limitation remains in the diversity of the exploiters. Complex environments such as MicroRTS contain many more weaknesses that can be exploited using only slightly different strategies than there are historical exploiters that can be created during training. As a result, the exploiters will most likely cover only those weaknesses that are exploitable from strategies similar to their initial agents policies. We therefore hypothesize that the diversity of the initial agents is crucial for the diversity of the exploiters and possibly more important than the initial agents' strength. This is because the initial policies strongly influence which regions of the strategy space are likely to be reached during training. This was probably less problematic in AlphaStar, because its initial policy was pre-trained by imitation learning on human replay data, which covered a much broader range of strategies. As a consequence, the initial policy was likely competent across a broader range of strategic situations than the Base Agent used in this thesis, which was trained with BC and PPO against only two built-in bots and was therefore biased towards a comparatively narrow strategy set. This broader

initialization presumably made it easier for exploiters to reach and exploit a more diverse set of strategies. A more diverse set of exploiters would improve the Main Agent's opponent diversity and thereby its generalization. The diversity of our Base Agent is further constrained by the fact that, during BC and PPO, some possible opponents must be excluded from training in order to obtain a more robust evaluation, while many strong agents are too slow to be practical training opponents.

Future Work: Map Diversity. Changing maps is already possible in MicroRTS-Py. Automatically generating new maps with different layouts and resource distributions and roughly equal chances of winning for both players is an interesting direction for future work, which could encourage the agent to learn more generalizable strategies and to adapt to different situations. This would also help to mitigate the problem of insufficient diversity if the maps force the agent to learn different strategies, for example by starting the training on maps that already contain units of different unit types, which can encourage different strategies. Another promising idea would be to train an agent to generate symmetrical maps on which the learning agent is weaker than the bot opponent during PPO or LT. The learning agent trained on those maps could then be used as a diverse initial agent for the exploiters and the Main Agent.

5. Conclusion

In this thesis, we implemented a modified LT method from AlphaStar [Vinyals et al., 2019] in combination with PPO in the MicroRTS environment, using a Base Agent trained on BC and PPO as the initial agent for the Main Agent and the exploiters, to improve the Main Agent's win, draw and loss rate against various opponents. We also expected to mitigate the need for many different bot agents to achieve diversity, by using exploiters to create a diverse set of opponents for the Main Agent.

As shown in the results, LT improves the Main Agent's performance against a wide range of opponents, including the built-in bots and the Base Agent used during training, and generalizes to other bots that were not used in training. This suggests the viability of the LT and PPO combination for improving the performance of an Base Agent trained on BC and PPO in MicroRTS. However, the diversity of the exploiters initial agents seems to be a crucial factorThis solution has its own limitations, such as not guaranteeing that all relevant clusters in the reward landscape are covered by the initial agents and complicating the training process by requiring the training of multiple initial agents. Further more this means that for this approach, diverse bot agents for BC and PPO training are still just as important or even more important for the performance of the Main Agent. While exploiters can provide some diversity and mitigate the need for many variations of bot agents with different but similar strategies, they cannot fully replace the need for diverse bot agents in this training approach. This is because more diverse strategies are likely to be more difficult to find for the exploiters and therefore may not be represented. However, we presume that replacing our current BC and PPO approach with an approach focusing on learning diverse strategies from the start, could further increase the Main Agent's performance, especially its generalization and robustness to a wider range of opponents. It would also reduce the need for many different bot agents for training. The map diversity method mentioned in the future work section could be a promising approach to this issue. Human imitation learning, as used in AlphaStar, seems to be a difficult approach for MicroRTS, because of the data availability and quality.

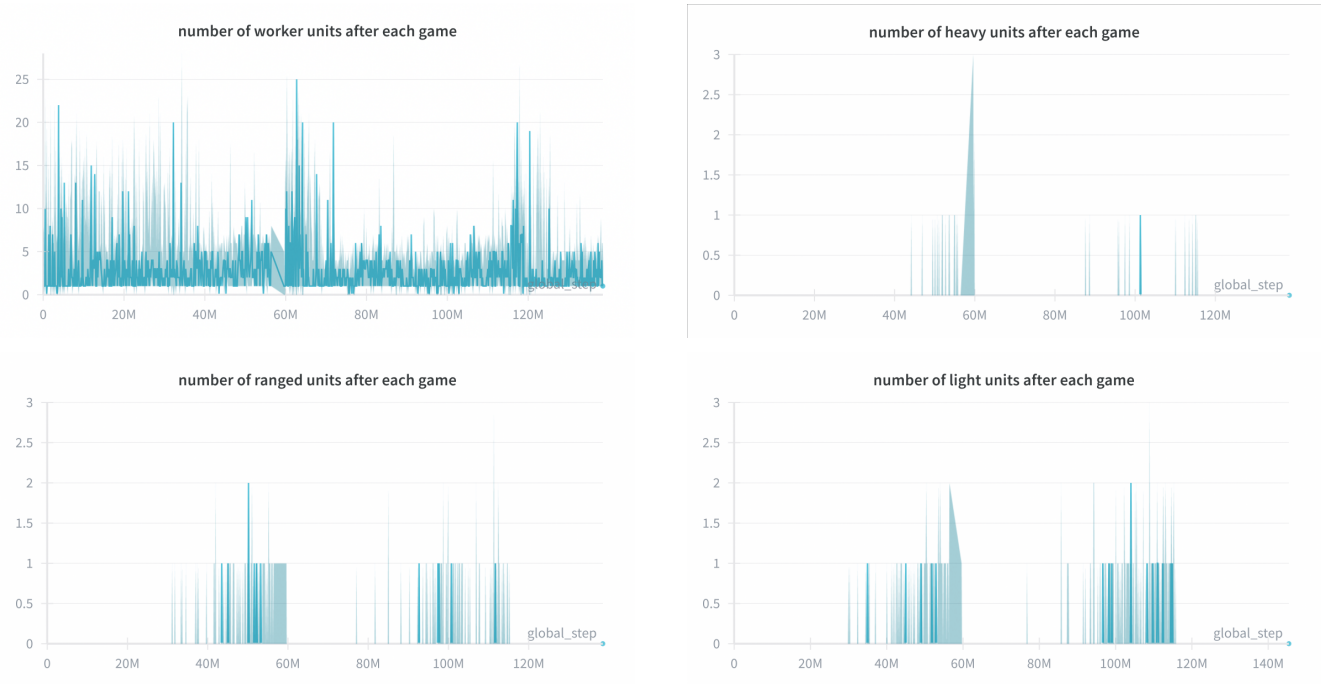

Figure 6. Number of units after each game for the agents initialized with worker focused (shown are 2 full reset cycles).

Table 2. Checkpoint amount per initial agent type and the approximate total games that exploiters trained with them.

Agent type	**League Exploiter**	**Main Exploiter**	**avg. win rate**	**total exploiter games**
Base Agent	7	2	4	50000
worker_focused	0	0	2	11000
ranged_focused	0	0	1	387
light_focused	0	0	2	14000
Total	7	2	9	75387

Table 3. Base Agent evaluation

Opponent	Games	Wins	Draws	Losses	Win rate	Draw rate	Loss rate
bots used in training							
coacAI	300	288	6	6	0.96	0.02	0.02
mayari	300	202	16	82	0.673	0.053	0.273
Built-in bots							
workerRushAI	300	300	0	0	1	0	0
passiveAI	300	299	1	0	0.997	0.003	0
lightRushAI	300	294	0	6	0.98	0	0.02
randomAI	300	297	3	0	0.99	0.01	0
randomBiasedAI	300	237	63	0	0.79	0.21	0
Other bots							
rojo	300	243	41	16	0.81	0.137	0.053
mixedBot	300	130	67	103	0.433	0.223	0.343
izanagi	300	215	36	49	0.717	0.12	0.163
tiamat	300	279	0	21	0.93	0	0.07
droplet	300	203	70	27	0.677	0.233	0.09
guidedRojoA3N	300	224	71	5	0.747	0.237	0.017
naiveMCTSAI	300	2	290	8	0.007	0.967	0.027
total	4200	3213	664	323	0.765	0.158	0.077

Table 4. Main Agent evaluation

Opponent	Games	Wins	Draws	Losses	Win rate	Draw rate	Loss rate
bots used in training							
BaseAgent	100	89	9	2	0.89	0.09	0.02
coacAI	100	100	0	0	1	0	0
mayari	100	100	0	0	1	0	0
Built-in bots							
workerRushAI	100	35	10	55	0.35	0.1	0.55
passiveAI	100	100	0	0	1	0	0
lightRushAI	100	100	0	0	1	0	0
randomAI	100	100	0	0	1	0	0
randomBiasedAI	100	97	3	0	0.97	0.03	0
Other bots							
rojo	100	100	0	0	1	0	0
mixedBot	100	57	23	20	0.57	0.23	0.2
izanagi	100	70	23	7	0.7	0.23	0.07
tiamat	100	99	0	1	0.99	0	0.01
droplet	100	62	16	22	0.62	0.16	0.22
guidedRojoA3N	100	87	13	0	0.87	0.13	0
naiveMCTSAI	100	2	98	0	0.02	0.98	0
total	1500	1198	195	107	0.799	0.130	0.071

Table 5. Difference in evaluation results between the Main Agent and the Base Agent

Opponent	Win rate	Draw rate	Loss rate
bots used in training			
coacAI	0.04	-0.02	-0.02
mayari	0.327	-0.053	-0.273
Built-in bots			
workerRushAI	-0.65	0.1	0.55
passiveAI	0.003	-0.003	0
lightRushAI	0.02	0	-0.02
randomAI	0.01	-0.01	0
randomBiasedAI	0.18	-0.18	0
Other bots			
rojo	0.19	-0.137	-0.053
mixedBot	0.137	0.007	-0.143
izanagi	-0.017	0.11	-0.093
tiamat	0.06	0	-0.06
droplet	-0.057	-0.073	0.13
guidedRojoA3N	0.123	-0.107	-0.017
naiveMCTSAI	0.013	0.013	-0.027

Table 6. Example for Main Agent without unit focus evaluation

Opponent	Games	Wins	Draws	Losses	Win rate	Draw rate	Loss rate
bots used in training							
BaseAgent	100	85	15	0	0.85	0.15	0
coacAI	100	100	0	0	1	0	0
mayari	100	100	0	0	1	0	0
Built-in bots							
workerRushAI	100	31	6	63	0.31	0.06	0.63
passiveAI	100	100	0	0	1	0	0
lightRushAI	100	100	0	0	1	0	0
randomAI	100	100	0	0	1	0	0
randomBiasedAI	100	91	9	0	0.91	0.09	0
Other bots							
rojo	100	100	0	0	1	0	0
mixedBot	100	45	22	33	0.45	0.22	0.33
izanagi	100	68	17	15	0.68	0.17	0.15
tiamat	100	94	0	6	0.94	0	0.06
droplet	100	45	33	22	0.45	0.33	0.22
guidedRojoA3N	100	64	36	0	0.64	0.36	0
naiveMCTSAI	100	0	100	0	0	1	0
total	1500	1123	238	139	0.749	0.159	0.093

Table 7. Evaluation after PPO finetuning

Opponent	Games	Wins	Draws	Losses	Win rate	Draw rate	Loss rate
bots used in training							
BaseAgent	100	92	8	0	0.92	0.08	0
coacAI	100	100	0	0	1	0	0
mayari	100	100	0	0	1	0	0
workerRushAI	100	100	0	0	1	0	0
Built-in bots							
passiveAI	100	100	0	0	1	0	0
lightRushAI	100	100	0	0	1	0	0
randomAI	100	100	0	0	1	0	0
randomBiasedAI	100	98	2	0	0.98	0.02	0
Other bots							
rojo	100	100	0	0	1	0	0
mixedBot	100	59	24	17	0.59	0.24	0.17
izanagi	100	68	29	3	0.68	0.29	0.03
tiamat	100	99	0	1	0.99	0	0.01
droplet	100	66	20	14	0.66	0.2	0.14
guidedRojoA3N	100	84	16	0	0.84	0.16	0
naiveMCTSAI	100	2	97	1	0.02	0.97	0.01
total	1500	1268	196	36	0.845	0.131	0.184

Table 8. Hyperparameters and control parameters used for the LT setup

Category	Parameter	Value	Description
LT Setup			
League	num_main_exploiters	4	Number of Main Exploiters
League	num_league_exploiters	2	Number of League Exploiters
League	selfplay_ready_save_interval	$5 \cdot 10^5$	Step interval for potential league checkpoints (checkpoint if the win rate is sufficiently high)
League	main_selfplay_save_interval	$6 \cdot 10^6$	Step interval for forced Main Agent checkpoints
League	main_exploiter_selfplay_save_interval	$9 \cdot 10^6$	Step interval for forced Main Exploiter checkpoints
League	league_exploiter_selfplay_save_interval	$9 \cdot 10^6$	Step interval for forced League Exploiter checkpoints
League	main_winrate_threshold	0,7	Win rate threshold for Main Agent checkpoints
League	main_exploiter_winrate_threshold	0,7	Win rate threshold for Main Exploiter checkpoints
League	league_exploiter_winrate_threshold	0,7	Win rate threshold for League Exploiter checkpoints
League	main_exploiter_vs_main_winrate_threshold	0,3	Threshold for Main Exploiters against the Main Agent
PFSP	pfsp_min_prob_factor	0,25	Minimum probability factor added to PFSP
PFSP	main_PFSP_prob	0,5	Probability of using PFSP with historicals for the Main Agent
PFSP	main_SP_prob	0,35	Probability of SP for the Main Agent
Environments and Training Scope			
Environments	num_bot_envs	5	Initial number of bot environments.
Environments	min_num_bot_envs	5	Lower bound for the number of bot environments
Environments	max_num_bot_envs	10	Upper bound for the number of bot environments
Environments	min_bot_winrate	0,955	Lower win rate threshold for bot environment adjustment (if crossed num_bot_envs == max_num_bot_envs)
Environments	max_bot_winrate	0,98	Upper win rate threshold for bot environment adjustment (if crossed num_bot_envs == min_num_bot_envs)
Environments	num_main_envs	25	Number of parallel environments for the Main Agent
Environments	num_envs_per_main_exploiters	25	Number of environments per Main Exploiter
Environments	num_envs_per_league_exploiters	25	Number of environments per League Exploiter
Training	total_timesteps	200 000 000	Total number of training steps
Training	checkpoint_end_buffer_steps	$3 \cdot 10^6$	Exploiters cannot checkpoint checkpoint_end_buffer_steps steps bevor reaching total_timesteps
Reward Function			
Reward	attack_reward_weight	0,0	Weight of the attack reward
Reward	rewardscore	0,002	Scaling factor of delta score.
PPO Hyperparameters (Main Agent)			
PPO Main Agent	PPO_learning_rate	$5 \cdot 10^{-5}$	Learning rate of the Main Agent
PPO Main Agent	num_steps	512	Number of steps per rollout
PPO Main Agent	gamma	1,0	Discount factor
PPO Main Agent	gae_lambda	0,95	Generalized Advantage Estimation (GAE) parameter
PPO Main Agent	ent_coef	0,01	Entropy regularization coefficient
PPO Main Agent	vf_coef	0,15	Weight of the value loss
PPO Main Agent	max_grad_norm	0,5	Gradient clipping threshold
PPO Main Agent	clip_coef	0,15	PPO clipping coefficient
PPO Main Agent	update_epochs	4	Number of update epochs per PPO cycle.
PPO Main Agent	kle_stop	true	Early stopping when KL divergence becomes too large
PPO Main Agent	kle_rollback	false	No rollback when the KL threshold is exceeded
PPO Main Agent	target_kl	0,01	Target KL divergence value
PPO Main Agent	kl_coeff	0,3	Coefficient of the KL penalty term
PPO Main Agent	norm_adv	true	Advantages are normalized
PPO Main Agent	anneal_lr	true	Learning rate annealing is enabled
PPO Main Agent	clip_vloss	true	Value loss clipping is enabled
PPO Hyperparameters (exploiters) (only differences to Main Agent hyperparameters)			
PPO Exploiter	exploiter_ent_coef	0,015	Entropy regularization coefficient of the exploiters
PPO Exploiter	exploiter_vf_coef	0,1	Weight of the value loss for the exploiters

Some Parameters (e. g. model paths, project names, or logging names) were omitted for clarity.

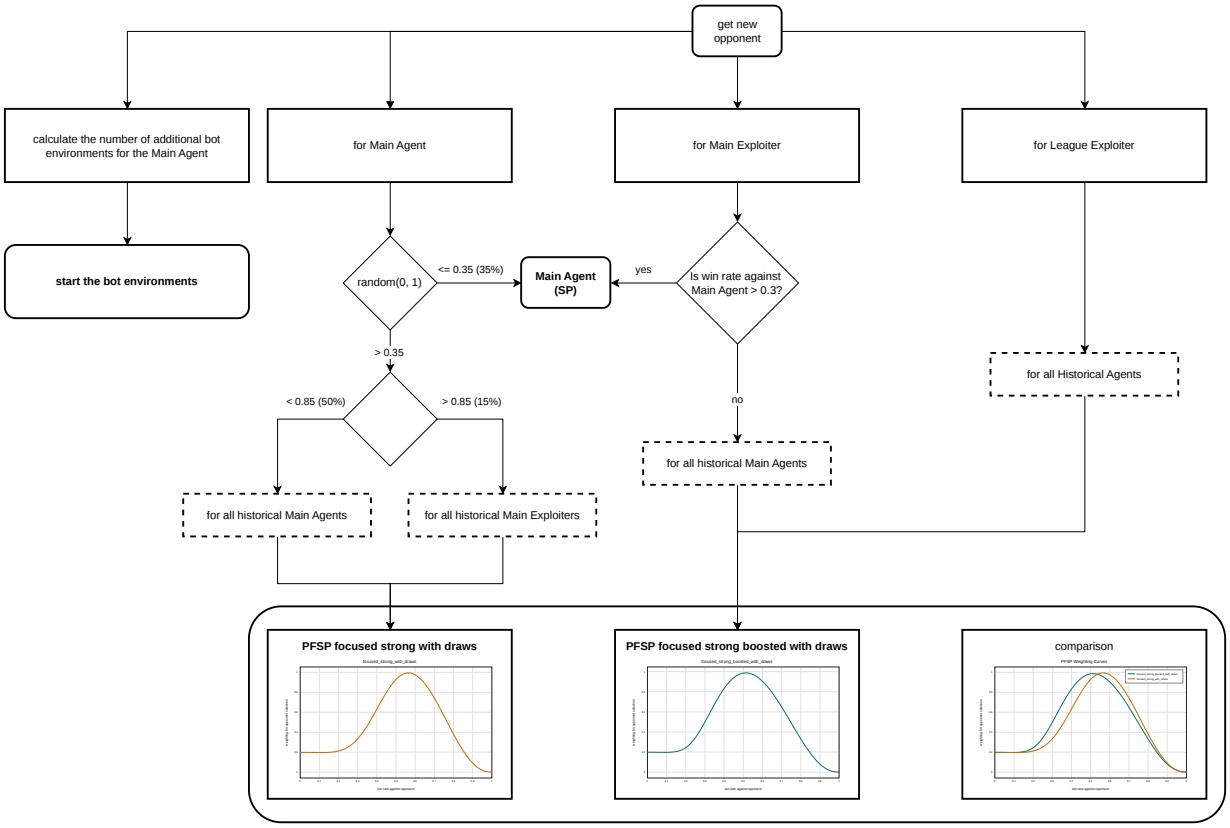

Figure 7. Matchmaking process for LT. Dashed boxes are candidate opponent sets, while the percentages on the arrows are fixed sampling probabilities. The PFSP curve gives the weighting for the opponent win rate.

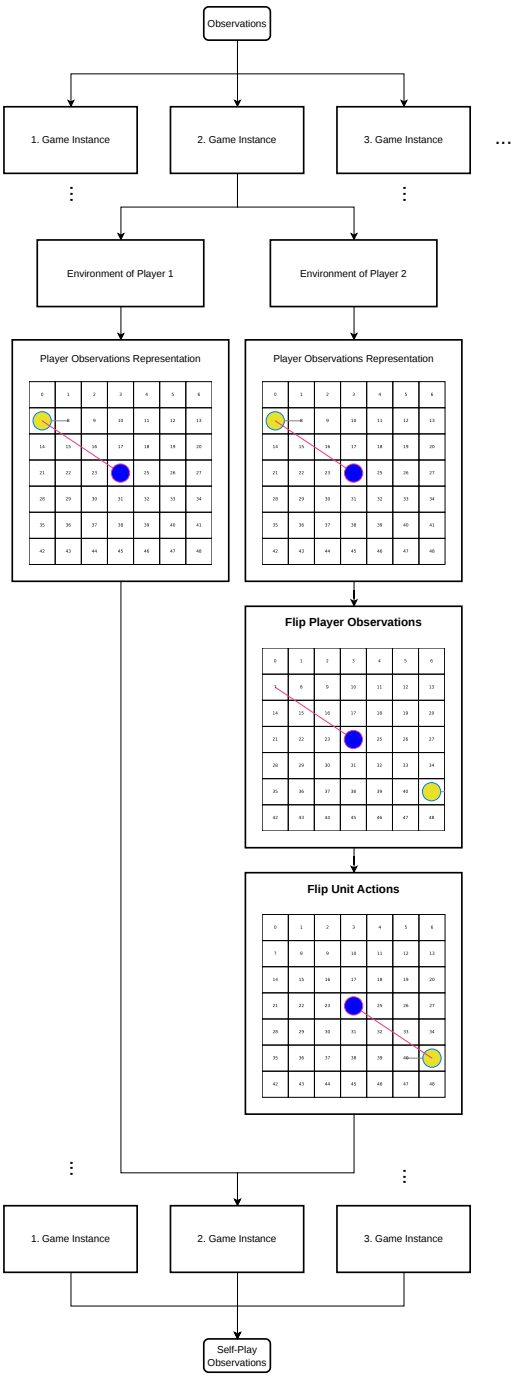

Figure 8. Observation adjustment for player 2 in SP. To rotate the observations, the GridNet representation was flipped for each player-2 environment. Afterwards, the unit actions in the observation were flipped individually.

References

[Dhariwal et al., 2017] Dhariwal, P., Hesse, C., Klimov, O., Nichol, A., Plappert, M., Radford, A., Schulman, J., Sidor, S., Wu, Y., and Zhokhov, P. (2017). Openai baselines. https://github.com/openai/baselines. 3

[Espeholt et al., 2018] Espeholt, L., Soyer, H., Munos, R., Simonyan, K., Mnih, V., Ward, T., Doron, Y., Firoiu, V., Harley, T., Dunning, I., Legg, S., and Kavukcuoglu, K. (2018). Impala: scalable distributed deep-rl with importance weighted actor-learner architectures. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, volume 80 of *Proceedings of Machine Learning Research*, pages 1406–1415. 3, 4

[Goodfriend, 2024] Goodfriend, S. (2024). Raisocketai: The first deep reinforcement learning agent to win the ieee microrts competition. In *2024 IEEE Conference on Games (CoG)*, pages 1–8. 3, 5, 6, 11

[Han et al., 2019] Han, L., Sun, P., Du, Y., Xiong, J., Wang, Q., Sun, X., Liu, H., and Zhang, T. (2019). Grid-wise control for multi-agent reinforcement learning in video game AI. In Chaudhuri, K. and Salakhutdinov, R., editors, *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 2576–2585. PMLR. 3, 5

[Heinrich et al., 2015] Heinrich, J., Lanctot, M., and Silver, D. (2015). Fictitious self-play in extensive-form games. In Bach, F. and Blei, D., editors, *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pages 805–813. 2, 6

[Huang et al., 2021] Huang, S., Ontanón, S., Bamford, C., and Grela, L. (2021). Gym-microrts: Toward affordable full game real-time strategy games research with deep reinforcement learning. *CoRR*, abs/2105.13807. 2, 3, 5, 6, 11

[Huft, 2025] Huft, M. (2025). Game playing agents in microrts. 2, 3, 5, 11, 12, 13

[Martin and Jhala, 2021] Martin, D. and Jhala, A. (2021). Beclone: Behavior cloning with inference for real-time strategy games. In *Joint Proceedings of the AIIDE 2021 Workshops*. 2, 4, 5

[Ontañón, 2013] Ontañón, S. (2013). The combinatorial multi-armed bandit problem and its application to real-time strategy games. In *Proceedings of the Ninth Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE)*, volume 9, pages 58–64. AAAI. 3, 11

[Schulman et al., 2017] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. (2017). Proximal policy optimization algorithms. *CoRR*, abs/1707.06347. 2, 5

[Silver et al., 2018] Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., Lanctot, M., Sifre, L., Kumaran, D., Graepel, T., Lillicrap, T., Simonyan, K., and Hassabis, D. (2018). A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. *Science*, 362:1140–1144. 6

[Sutton and Barto, 1998] Sutton, R. S. and Barto, A. G. (1998). *Reinforcement Learning: An Introduction*. The MIT Press. 4

[Torabi et al., 2018] Torabi, F., Warnell, G., and Stone, P. (2018). Behavioral cloning from observation. *arXiv preprint arXiv:1805.01954*. 2, 5

[Vinyals et al., 2019] Vinyals, O., Babuschkin, I., Czarnecki, W. M., Mathieu, M., Dudzik, A., Chung, J., Choi, D. H., Powell, R., Ewalds, T., Georgiev, P., Oh, J., Horgan, D., Kroiss, M., Danihelka, I., Huang, A., Sifre, L., Cai, T., Agapiou, J. P., Jaderberg, M., Vezhnevets, A. S., Leblond, R., Pohlen, T., Dalibard, V., Budden, D., Sulsky, Y., Molloy, J., Paine, T. L., Gulcehre, C., Wang, Z., Pfaff, T., Wu, Y., Ring, R., Yogatama, D., Wünsch, D., McKinney, K., Smith, O., Schaul, T., Lillicrap, T., Kavukcuoglu, K., Hassabis, D., Apps, C., and Silver, D. (2019). Grandmaster level in starcraft ii using multi-agent reinforcement learning. *Nature*, 575(7782):350–354. 2, 4, 6, 7, 8, 9, 13, 14

Erklärung

(???) GPT!! Hiermit versichere ich, dass ich die Bachelor-Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe.

Düsseldorf, den XXXXXXXXXXX

Name!!!